

this lack of efficiency exists in relatively simple tasks, how much exists when switching between highly creative work, such as a new system design, and a consultant's work proposal? The team swapping of matrix management can compound this with the loss of performance gains associated with team bonding.

Slack documents many of the problems of cutting staff 'overhead' and expecting managers to do their own administration work. The poor performances of various failing organizations are traced back to the overlooked causes of time-starved teams, stressed and fearful staff, and overstaffed projects.

Ninety years after the work of Fredrick Taylor (*The Principles of Scientific Management*), the modern workplace challenges the "Process Obsession" because it stifles effectiveness and empowerment. Command and control is not a creative environment for innovation. DeMarco reexamines the definition of quality and the motivation and effectiveness of quality programs. He also shines a bright light on the silliness of a blind adherence to a partial definition of quality that neglects broader concerns.

While *Slack* takes only a few hours to read, it will make you think for weeks. This is one of the best books on technology management in a long time. It recognizes the humanity of organizations while pointing the way to real effectiveness and efficiency. It challenges all the good ideas that have become movements in the hands of consultants by pointing back to the original intentions.

Just as with the *Treatise on Comedy* in *The Name of the Rose*, there will be some consultants that would rather burn this book than allow managers to hear its message: efficiency programs can and do harm organizational effectiveness. I will keep a tight hold on my copy and read chapters from it often.

Carol A. Long is a software developer and project manager working in a software process improvement group and searching for the mythical perfect project. Contact her at carollong@bcs.org.uk.

Tips to Improve Software Development

Anthony Akins

Agile Software Development by Alistair Cockburn, Addison-Wesley, Boston, 2001, ISBN 0-201-69969-9, 278 pp., US\$34.99.

Over the last couple of days, I've been trying to figure out how to describe *Agile Software Development* by Alistair Cockburn. It's not a textbook. It's not a cookbook. It's definitely not a silver bullet, guaranteed to solve all of your software development project problems. So, what is it? How about a multilayered collection of thoughts and ideas on how to adapt our approach to software development to fit the current situation and organization? That's a mouthful, but it's the most concise and accurate description I've been able to come up with.

Three main sections make up *Agile Software Development*. The first section is concerned with abstract thoughts concerning software development: how people communicate and how teams work and collaborate. The second section is more concrete and presents the concepts and elements of software methodologies. The third section focuses on agile software development: what it means to be agile, with principles and sug-

gestions for building your own agile development methodology.

Cockburn presents ideas and information for those new to software development, as well as to those who are "long in the tooth." Everyone from sponsors to project managers, technical leads, process and methodology designers, and software developers can learn and benefit from this book. He provides different lessons for different folks but continually focuses the reader's attention on two fundamental and daunting problems:

- If communication is fundamentally impossible, how can the people on a project manage it?
- If all people and all projects are different, how can we create any rules for productive projects?

The pessimistic among us might feel there is no hope. Truth is, we'll probably never get software development perfect, but we can get better, and for me, that's what *Agile Software Development* is all about: getting better at what we do through communication, collaboration, insight, study, practice, and experimentation.

Every chapter presents a collection of ideas, problems, and suggestions related to a particular aspect of software development. Each chapter ends with "What should I do tomorrow?" and helps the reader apply what they have read to their own project, work, and organization. Depending on your level of understanding, Cockburn might ask you to observe your own organization and compare what you see to what you read, start applying the concepts of what you read to your own work, or teach the concepts you read to others.

Applying the ideas raises your level of awareness of the issues the chapter covered and helps you see different ways to communicate, work as a team, and develop software. After my initial read, I came across three immediately useful ideas. The first was using the whiteboard as a way to force and focus design in the small. The second was using a reflection workshop as a simple and effective

"Agile Software Development" is not a textbook. It's not a cookbook. It's definitely not a silver bullet, guaranteed to solve all of your software development project problems. So, what is it?

tive way to encourage continuous lessons learned and improvements. Finally, the book made me think of ways to adapt a software methodology to the project, environment, and the organization (instead of the other way around).

Agile Software Development reinforced many of my fundamental beliefs about software development and at the same time exposed me to new concerns and ideas. An example of this comes from the section “Agile and Self Adapting,” where Cockburn identifies five sweet spots of effective software development:

1. *Two to eight people one room.* Face-to-face communication is the cheapest and most effective form of communication. Find a way to ensure people can communicate face to face whenever possible.
2. *Onsite usage experts.* They know what they want (or if they don’t they can probably recognize what they don’t want), so make every effort to give your team exposure to the people who will use the software. Successful projects find ways to get fast and effective user feedback from the beginning of the project to the end.
3. *One-month increments.* People are able to focus their efforts for about three months, but not much longer, so you don’t want increments longer than three months. You also want to allow enough time to get something meaningful done. One month is a good size, but adapt the duration of your increments to fit the situation.
4. *Fully automated regression tests.* This gives developers the confidence to make changes, because the automated tests will tell them if they broke something since the last time the tests ran. If you can’t fully automate regression, look for ways to provide feedback on software changes.
5. *Experienced developers.* People who know what they are doing are invaluable. If you need six experienced people but you can only get four, consider adding less experienced people, but realize you will need more than six (maybe 10, maybe more) to get the work done of six experienced people.

Can you use this book? If you’re happy with the state of software development in your organization, well, maybe not. You might want to read it to see if the ideas and principles documented in his book are similar to the ones you follow. For the rest of us, *Agile Software Development* provides ideas and guidance to open our eyes to more effective and efficient methods of developing software.

Anthony Akins is a developer and consultant at VLA Futures. Contact him at asakins@ureach.com

Demystifying Software Architecture

Deependra Moitra

Applied Software Architecture by Christine Hofmeister, Robert Nord, and Dilip Soni, Addison-Wesley, Boston, 2000, 422 pp., US\$44.95.

With software fast becoming central to most products and services and getting increasingly complex, amalgamated with constantly changing technologies, the architecture of software systems assumes paramount importance. If you are looking for help and guidance in gaining a solid footing in software architecture, then look no further. *Applied Software Architecture* is a nicely conceptualized and well-written book, full of practical insights and real-life examples.

This book will be of great use to many and particularly good for novice and budding software architects. Practicing architects will benefit from access to the holistic approach presented in this book. Software engineers planning to become software architects will find it a good self-help guide. It is also an excellent resource for corporate universities in training their engineers on software architecture, and it will aid university students and researchers wishing to dig deeper into the discipline and its relationship to software quality. Managers keen to

understand what considerations are involved in architecting a software system will also find this a useful addition to their bookshelf. There are various books on the subject, but this one is particularly noteworthy because of its pragmatic coverage, insightful treatment, and detailed real-world cases.

Backed by the authors’ years of industry experience, the book clearly explains how to think about software architecture, how to evolve and design a competitive software architecture, and the role of architecture in the development process. It teaches you rules and techniques and shares with you a proven, structured process for designing winning architectural solutions. It shows you how to trace influencing factors and requirements through the architecture, sequence and organize design activities, make architectural design tradeoffs, support system performance and reliability requirements, and align architecture with business goals, among other things. All the illustrations are based on UML notations, and each chapter provides pointers for additional reading. The book has a good and relevant bibliography, a well-presented glossary, and numerous useful tables and figures.

Four views, one integrated approach

First, the authors give an overview of how to structure architectural design tasks in general, then they explain how to define these tasks and apply them to software architecture design. The entire book and its approach are based on four software architecture views: conceptual, modular, execution-oriented, and code-based. The authors use their experiences architecting a large industrial image acquisition and processing system to illustrate and support the concepts and techniques introduced in each architectural view.

Next, the authors present the most critical aspect of architecting software systems, global analysis an integrated framework and approach for looking at software architecture. This discussion teaches you how to analyze organizational, technological, and product